

COGNITIVE RADIO SIMULATION - MACHINE LEARNING FUNDAMENTALS Q&A

Ten core ML concepts mapped to what is implemented in this Cognitive Radio software simulation. Each question is numbered; each answer uses the same question number.

Primary code references: `cr_sim/ml_models.py`, `cr_sim/cr_engine.py`, `cr_sim/channel.py`, `cr_sim/orchestrator.py`, `app.py`

1. Question: Data first - Where does the data for this model come from, and is it enough, clean, and representative?

1. Answer: In this simulation, training data is generated synthetically inside the same channel / ECM simulator (`ChannelRecommenderML.train_synthetic` in `ml_models.py`). About 800-1200 labelled cases are created per training run, covering jam profiles NONE, SPOT, MULTI_SPOT, SWEEP, and BARRAGE, with varied transmit power and active carrier. Features are numeric and cleaned by construction (no missing RF packets). The data is representative of the software contested-spectrum scenarios in this demo; it is not yet field-recorded I/Q from eADM hardware. For the PC demonstration this is sufficient; for deployed EW learning, later work would add real sensed telemetry while keeping the same feature interface.

2. Question: Problem type - Is this classification, regression, clustering, or generation?

2. Answer: This is a supervised multi-class classification problem. The model predicts one discrete action class: recommended channel id 0 ... N-1, or a special class -1 meaning INCREASE_POWER. It does not regress a continuous SINR estimate, does not cluster threats, and does not generate waveforms. This choice matches the Cognitive Radio actuation need: pick a channel or advise a power increase.

3. Question: Features - What input variables does the model learn from, and why do they matter?

3. Answer: The feature vector length is $3N + 2$. For each of N channels the model receives SINR (dB), a state encoding (FREE=2, DEGRADED=1, BLOCKED=0), and measured jam power; it also receives current transmit power and remaining power headroom ($P_{\text{max}} - P_{\text{current}}$). For the default $N = 8$ this is 26 features, matching the UI 'Input Vector: 26 normalized sensing features.' Features are standardised with `StandardScaler` before MLP inference. Feature quality matters more than model size here: if sensing states and jam power are wrong, the recommender cannot propose safe channels.

4. Question: Training vs testing - How is data split so the model is checked on unseen cases?

4. Answer: Today the demonstrator trains on the full synthetic set and reports training accuracy via `sklearn.model.score(X_train, y_train)` after fit. A formal held-out test split is not yet enforced in `train_synthetic` (honest limitation for scientific review). Generalisation is checked operationally in the live UI on new jam scenarios the operator creates (Spot, Multi, Sweep, Barrage) that were not a single frozen test file. Recommended next hardening: split synthetic data into train/validation/test (for example 70/15/15) and report test accuracy / confusion matrix beside train accuracy.

5. Question: Algorithm / model - What mathematical structure learns the patterns?

5. Answer: The model is a feed-forward Multi-Layer Perceptron (MLP) classifier from `scikit-learn`: `MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu', solver='adam', max_iter=400, random_state=seed)`. Two hidden layers (64 and 32 neurons) map the feature vector to class logits over channels plus power-increase. This is intentionally a small, inspectable network suitable for a transparent defence software demo, not a large deep spectrogram model.

6. Question: Loss function - How does the model measure how wrong its predictions are?

6. Answer: For this multi-class MLP classifier, training minimises a cross-entropy / log-loss style objective over the predicted class probabilities (scikit-learn MLP classification loss). In plain terms: if the true label is 'use CH2' and the model assigns low probability to CH2, the loss is high; training pushes probability mass toward the correct class. Softmax probabilities shown in the UI (and Stage 2 confidence) come from this probabilistic classification head.

7. Question: Optimization - How does the model adjust its internal weights?

7. Answer: Weights are updated iteratively by the Adam optimiser (solver='adam'), which is an adaptive gradient-based method. Each training iteration reduces the classification loss on the synthetic samples until convergence or max_iter is reached. A fixed random seed makes training repeatable for demos. After training, inference is a single forward pass: scale features -> MLP -> class prediction + probabilities.

8. Question: Evaluation metrics - How do we judge whether the model is good?

8. Answer: The UI / training return currently emphasises classification accuracy on the training synthetic set (train_accuracy). Live demo evaluation is qualitative and scenario-based: correct avoidance of BLOCKED channels, sensible alternate FREE channel under Spot/Multi jam, INCREASE_POWER under Barrage when headroom remains, and safety overrides when ML proposes an unsafe channel. For a stronger scientific scorecard, add precision/recall/F1 per class, confusion matrix, and held-out test accuracy; for EW ops, also track mission metrics such as fraction of steps on FREE channels and time-to-escape after jam onset.

9. Question: Overfitting vs underfitting - How are these risks handled in this design?

9. Answer: Underfitting risk is reduced by using a non-linear MLP (64x32) rather than a single linear rule, and by training across diverse jam profiles and powers. Overfitting risk exists because current scoring is mainly on training data and the labeler is itself a rule-based expert function - so the MLP can largely emulate that expert. Mitigation in this architecture is mission-critical: the Hybrid Safety Gate vetoes BLOCKED or otherwise unsafe proposals even if the neural net is overconfident. Also, features are low-dimensional and standardised, and the network is small. Stronger ML hygiene later: held-out test set, early stopping, and comparing RULES vs ML vs HYBRID on fixed scenario suites.

10. Question: Deployment & monitoring - How is the trained model used in operation, and what would monitoring mean here?

10. Answer: In this project, deployment is the Streamlit Cognitive Radio simulation: after startup/reset/apply-config training, the MLP is used each simulation step inside CognitiveRadioEngine under ML or HYBRID mode, with proposals shown in the Transmitter Attached ML Policy card and Stage 2 of the explainability trace. Continuous monitoring in the demo sense is the live FREE/DEGRADED/BLOCKED map, confidence, dispatched action, and safety-override messages. For a later eADM / SDR deployment, monitoring would track data drift (new ECM behaviours), confidence calibration, override rate, and link outcomes, then trigger retraining when sensed distributions diverge from the synthetic training regime.

Closing note for scientists

1. This codebase implements an explainable ML loop for channel / power classification on synthetic contested-spectrum features.
2. The main scientific gap to call out honestly is formal train/test split reporting, which should be added before claiming strong generalisation metrics.
3. Defence assurance in this design does not rely on ML alone: hybrid safety containment remains mandatory around the neural proposal.

Prepared by: eAge Innovations Technical Engineering Team

For scientific evaluation by: Shri S. K. Sir & Senior Defence Scientists